DATA SCIENCE PROGRAM

INDIVIDUAL REPORT – SPRING 2026

Multimodal Techniques for Financial Time-Series Forecasting

Pranav Dhawan

April 2026

Abstract

Accurate stock price forecasting requires integrating heterogeneous signals ranging from raw price histories to unstructured financial news and structured corporate filings. Recent multimodal datasets such as FinMultiTime enable cross-modal financial analysis, but the comparative performance of different model architectures on such data remains underexplored. We present a systematic benchmarking study on S&P 500 constituents using three complementary modalities: historical price time series, financial news sentiment, and structured fundamental features from SEC XBRL filings. We compare unimodal baselines (LSTM, PatchTST, TSMixer, TCN-Transformer, TFT) against pretrained foundation models (Chronos T5-Large, Chronos-2), multimodal fusion architectures (TFT-FinBERT, TCN-FinBERT, ChronosFinBert, FinBERT+CNN+Transformer), and graph neural networks (GATv2, EvolveGCN). Our central finding is that the choice of fusion mechanism dominates the choice of temporal encoder: cross-attention and encoder-decoder fusion between a temporal encoder and FinBERT improve on unimodal baselines, while concatenation-based late fusion underperforms them due to modality imbalance in the shared bottleneck. Graph neural networks offer complementary cross-sectional structure that sequence models cannot represent. These findings provide practical guidance for designing multimodal financial forecasting systems at scale.

Contents

1 Introduction **3**

2 Problem Statement **3**

3 Dataset **3**

3.1 Stock Price Data 4

3.2 Financial News Data 4

3.3 Financial Reports & Earnings Data 4

3.4 Sector Relational Data 4

4 My Contributions **5**

4.1 Dataset Setup 5

4.2 Dataset Construction Pipeline 5

4.3 SEC XBRL Preprocessing 6

4.4 Experiments 6

4.4.1 Baseline Models 6

4.5 Chronos Models 9

4.6 Graph Models 10

5 Results and Findings **11**

6 Conclusion and Future Work **11**

References **13**

1 Introduction

Financial time-series forecasting sits at the intersection of economics, statistics, and machine learning. Traditional approaches relying solely on historical price data often fail to capture the abrupt, sentiment-driven dynamics of real-world markets. Financial decision-making inherently depends on heterogeneous sources — news, corporate disclosures, and market signals — and recent advances in deep learning have enabled meaningful extraction of signals from all of these.

Datasets such as FinMultiTime [1] have accelerated this direction by aligning text, tables, and time series into unified forecasting frameworks. However, how different architectures interact with such heterogeneous data remains underexplored. This project addresses that gap through systematic benchmarking across modalities and model families.

2 Problem Statement

This project investigates: *How effectively can multimodal deep learning architectures combine historical price time series, financial news sentiment, and structured financial features to improve short-horizon stock price forecasting on S&P 500 equities?*

Formally, let $\mathbf{x} = \{x_{t-T+1}, \ldots, x_t\}$ denote a lookback window of historical close prices, where T is the input length and x_t is the closing price on day t. The forecasting objective is to predict the next H trading-day close prices,

$$\mathbf{y} = \{x_{t+1}, \ldots, x_{t+H}\}.$$

Across experiments, $H = 5$ and $T \in \{30, 60, 90\}$ depending on the architecture.

3 Dataset

All experiments use the S&P 500 Multimodal Financial Dataset [1], a large-scale benchmark integrating financial news, reports, chart images, and stock price time series across U.S. (S&P 500) and Chinese (HS300) markets. We construct a reduced, task-specific version by excluding image data and HS300 samples, focusing solely on structured and textual signals from the U.S. market.

Dataset Type	**Size**	**Records**	**Frequency Count**	**Frequency**
Semi-Annual Table	84.04 GB	2,676	8,028	Semi-Annual
Quarterly/Annual Timeseries	1.83 GB	4,213	4,213	Quarterly / Annual
Daily Text	14.1 GB	4,694	3,351,852	Daily
Total (All)	108.64 GB	—	3,559,440	—

Table 1: Summary of the reduced S&P 500 multimodal dataset used in this study.

3.1 Stock Price Data

The time-series source contains daily OHLCV (Open, High, Low, Close, Volume) records per ticker. Dividends, stock splits, and capital gains were excluded due to inconsistent coverage. Data were cleaned, chronologically sorted, and transformed into model-ready representations comprising 21 features per timestep, including log returns, rolling summary statistics, and technical indicators.

3.2 Financial News Data

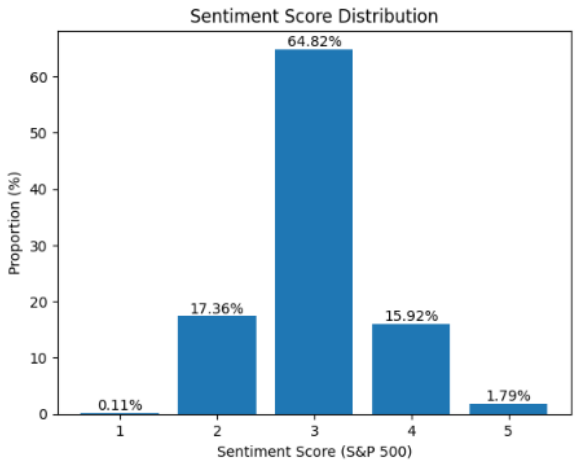

Figure 1: Distribution of sentiment scores in the text modality.

3.3 Financial Reports & Earnings Data

Structured financial indicators were derived from company filings, including key variables related to profitability, liquidity, and capital structure, such as cash flow, equity, and earnings. These variables were aligned with reporting periods and forward-filled to match trading days so that fundamental signals could be synchronized with daily market observations.

SEC Fundamental Data Preprocessing. Structured financial indicators were derived from company filings, covering profitability, liquidity, and capital structure (e.g., cash flow, equity, earnings). These were aligned with reporting periods and forward-filled to match trading days, synchronizing fundamental signals with daily observations.

3.4 Sector Relational Data

Each ticker was associated with its GICS sector and used to construct sector-based relational edges in graph models, enabling the pipeline to represent shared industry exposure and sector-level co-movement.

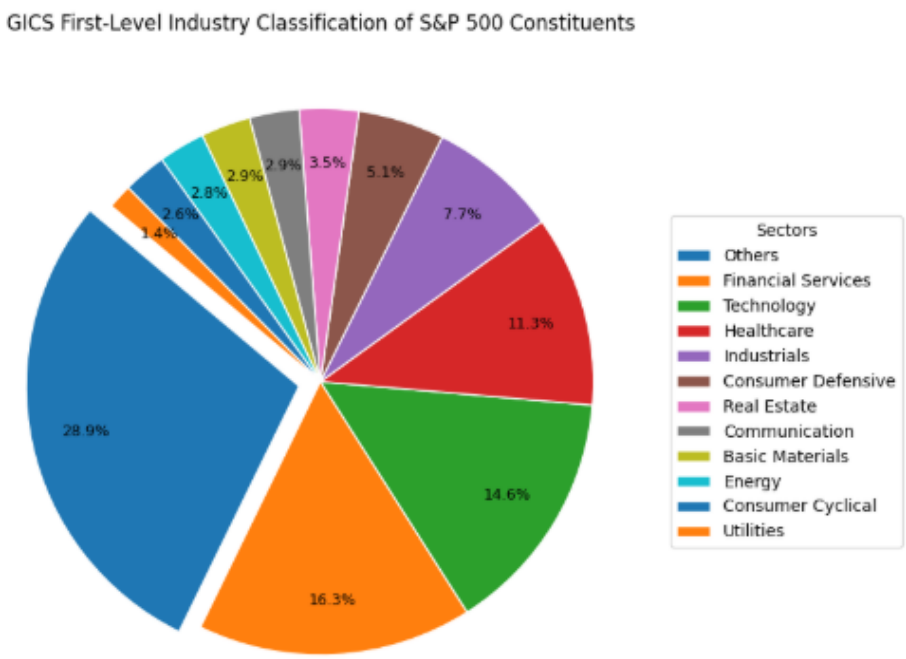

Figure 2: Distribution of stocks across GICS sectors in the relational metadata.

4 My Contributions

4.1 Dataset Setup

The file `dir_setup.sh` is a Bash automation script that downloads the FinMultiTime dataset from Hugging Face and prepares it for analysis. It skips image data, extracts only S&P 500 components, cleans up unnecessary files, and restructures the directory. Specifically, it: downloads the dataset excluding images via `huggingface_hub`; extracts only S&P 500-related zip files; deletes remaining zips to save space; renames the output directory to `data/`; removes `hs300stock_data_description.csv`; and flattens folder structure for simplicity.

4.2 Dataset Construction Pipeline

The multimodal sample generation process was implemented in `baseline_dataset_creation.py`. The pipeline loaded each ticker's OHLCV file, filtered the date range, and retained an ordered daily table as the temporal backbone. It incorporated tabular financial reporting data via `load_financial_jsonl` and `extract_financial_features`, extracting over 30 US-GAAP metrics (e.g., Assets, Liabilities, StockholdersEquity, NetCashProvidedByUsedInOperatingActivities). Since filings are quarterly, they were merged onto the daily calendar using `pd.merge_asof` with backward alignment.

News data were processed at the article level and aggregated daily: JSONL records were grouped

by date, summary variables (article count, average length) were computed, and raw texts were preserved for embedding-based models. These aggregates were merged into the daily table with a left join.

Once aligned, the pipeline generated supervised samples via a sliding window (history 30, horizon 5, stride 1). Each sample stored observation dates, daily article lists, time-series dictionaries, financial metric dictionaries, a GICS sector label, and a five-step target. Outputs were written to per-ticker JSONL files in `baseline_data/` with a `ticker2id.json` mapping for consistent indexing.

4.3 SEC XBRL Preprocessing

The SEC preprocessing pipeline consumed per-ticker JSON statement files from `data/sp500_table/<ticker>/*.json`. Each file corresponded to a financial statement (balance sheet, cash flow, equity) and contained nested US-GAAP facts.

The pipeline ran five stages: (1) **Extraction** — data points were pulled from the first filing entry and flattened into a long-format table with one row per metric-period observation; (2) **Metric-level deduplication** — for each `(metric, period_end)` pair, a canonical entry was selected preferring fiscal-year match, then ranked `10-K` > `10-K/A` > `10-Q` > `10-Q/A`, with ties resolved by most-recent filing; (3) **Filing-level deduplication** — one canonical period per filed date was selected by highest metric count, then latest period-end; (4) **Metadata selection** — one metadata record per `(filed_date, period_end)` was retained using the same priority logic; (5) **Pivoting** — data were reshaped from long to wide format with all extracted metric columns prefixed `sec_` to avoid naming collisions.

The output was one CSV per ticker, sorted by date, containing `date`, `period_end_date`, `fiscal_year`, `fiscal_period`, `form`, `ticker`, and `sec_<MetricName>` columns.

4.4 Experiments

4.4.1 Baseline Models

The baseline stage evaluated architectures with complementary inductive biases for short-horizon financial prediction. All baseline models were trained for five-step-ahead log-return prediction; exact input horizon and feature specifications varied by implementation.

GRU A GRU model was used as the first sequential baseline. It consumed a 60-day lookback of six engineered features and predicted all five future log returns in a single forward pass (MIMO design), with 99,493 trainable parameters. Three unidirectional GRU layers were stacked:

$$\text{GRU}(6\to128) \to \text{Dropout}(0.3) \to \text{GRU}(128\to64) \to \text{Dropout}(0.2) \\ \to \text{GRU}(64\to32) \to \text{Dropout}(0.1),$$

followed by an MLP head Linear(32→16) → GELU → Linear(16→5).

Six input features were constructed: daily log return, log volume change, high-low spread, 5-day rolling volatility, 20-day rolling volatility, and RSI(14). A `StandardScaler` was fit only on the

training segment. Training used Adam (lr $= 10^{-3}$, weight decay 10^{-4}), gradient clipping 1.0, batch size 128, Huber loss, and early stopping (patience 15).

LSTM The LSTM [2] extended the recurrent baseline with a memory cell designed to mitigate vanishing gradients. Configured with 26 input features, a 60-day lookback, a 5-step horizon, and 129,861 parameters, it used a two-layer stacked architecture:

$$\text{LSTM}(26\to128) \to \text{Dropout}(0.2) \to \text{LSTM}(128\to64) \to \text{Linear}(64\to5).$$

The 26-dimensional feature set included log returns, price-structure variables, volume features, rolling statistics, RSI(14), ATR(14), MACD, Bollinger width, and lagged returns at lags 1, 2, 3, 5. The training objective combined Huber loss with an anti-zero penalty. Average test metrics on standardized and original price scales are shown in Table 2.

Metric	Std. Scale	Log-ret Scale
MAE	1.0880	0.02939
MSE	1.9723	0.001439
RMSE	1.4043	0.03794
MAPE	319.53%	214.03%

Table 2: LSTM Metrics.

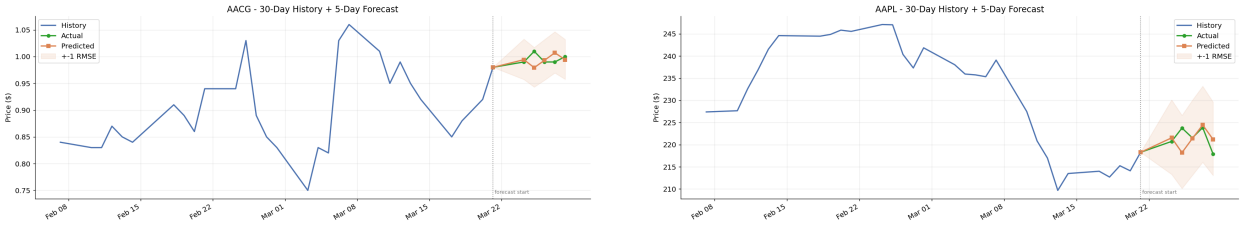

Figure 3: Representative LSTM forecast examples for AACG and AAPL.

PatchTST A PatchTST-style [3] multivariate model was trained and evaluated. Input shape was `(B, 60, 12)` with a 12-channel feature set (log return, intraday ranges, log volume, volume ratio, rolling statistics, RSI, MACD).

With context length 60, patch length 8, stride 4, each channel produced 14 patches. Patches were projected to 128 dimensions, combined with positional embeddings, and passed through three Transformer encoder blocks (8 heads, FF width 256), yielding 409,349 parameters. Only the `log_ret` channel's output was retained. Performance depended strongly on the target scale: standardized-target evaluation remained weak, whereas log-return-space evaluation produced materially lower errors and stronger validation directional accuracy. The full summary is reported in Table 3.

Metric	Std. Scale	Log-ret Scale
MAE	1.44	0.0147
MSE	3.04	0.0323
RMSE	2.36	0.0233

Table 3: PatchTST metrics.

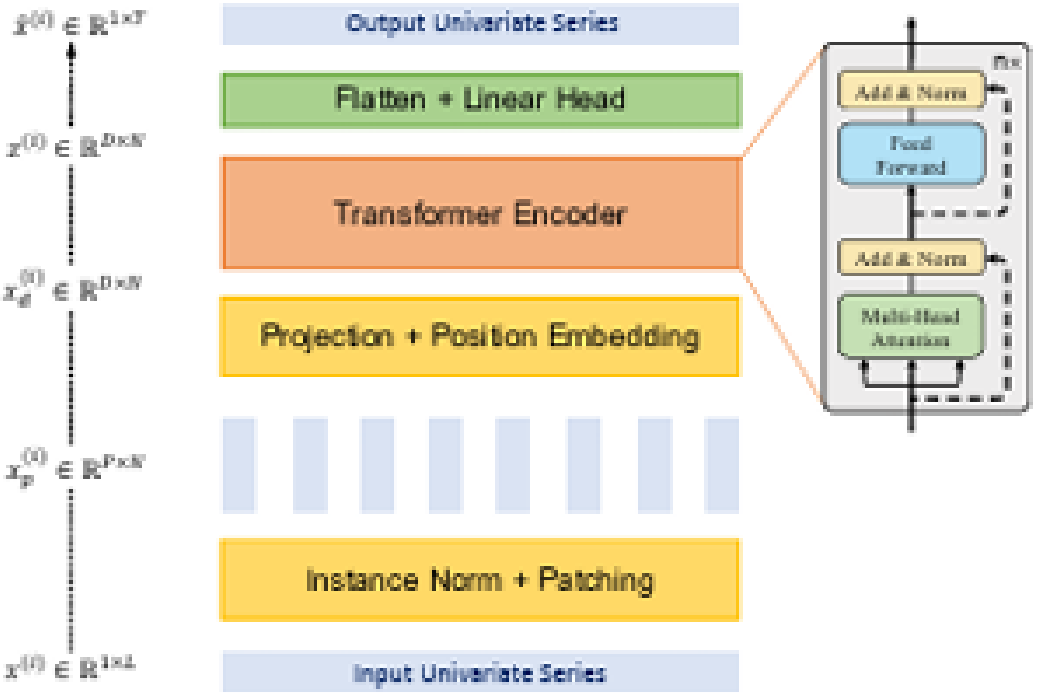

Figure 4: PatchTST architecture for multivariate stock forecasting.

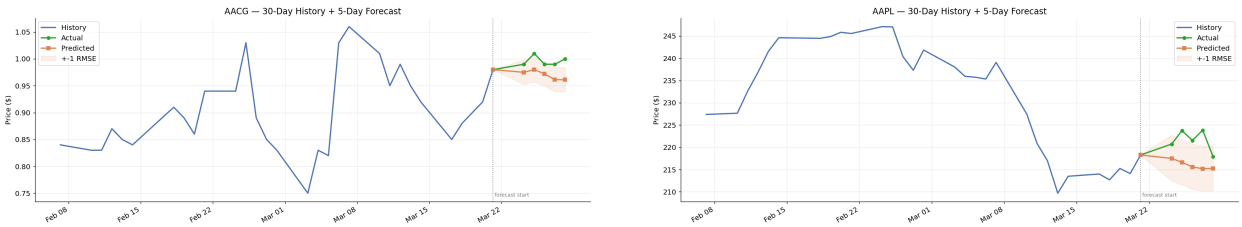

Figure 5: Representative PatchTST forecast examples for AACG and AAPL.

TSMixer TSMixer [4] was trained and evaluated as an all-MLP alternative. The model used four mixer blocks (FF width 128, dropout 0.1), input shape `[B, 60, 15]`, and output shape `[B, 5]`, with an 8-dimensional ticker embedding concatenated per timestep for stock-specific specialization. RevIN normalization and learnable temporal weights (ramp-initialized 0.5–1.0) were included. Training used AdamW (lr $= 10^{-3}$), cosine annealing, and a mixed MSE + quantile pinball loss ($q = 0.9$). Table 4 summarizes results. Absolute errors stayed nearly unchanged from validation to test, but reported IC was 0.0383 indicating that the model retained average fit.

Metric	Std. Scale	Log-ret Scale
MAE	3.9467	0.01474
MSE	162.45	0.0004963
RMSE	12.7896	0.02228

Table 4: TSMixer Metrics.

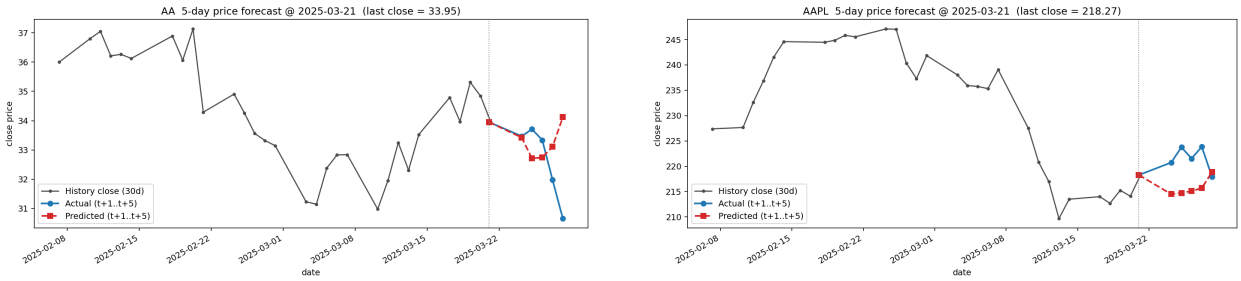

Figure 6: Representative TSMixer forecast examples for AA and AAPL.

4.5 Chronos Models

Chronos 2 Chronos 2 [6] was evaluated as a zero-shot pipeline using `amazon/chronos-2`. Context was capped at 2,048 timesteps. Results on a 20% sampled subset (seed 42) are shown in Table 5. A MASE of 0.6757 indicates that the zero-shot model outperformed a naive seasonal baseline even without task-specific adaptation.

Metric	Std. Scale	Log-ret Scale
MAE	1.1954	0.01758
MSE	19.0126	0.001736
RMSE	1.3505	0.02176

Table 5: Chronos 2 Metrics.

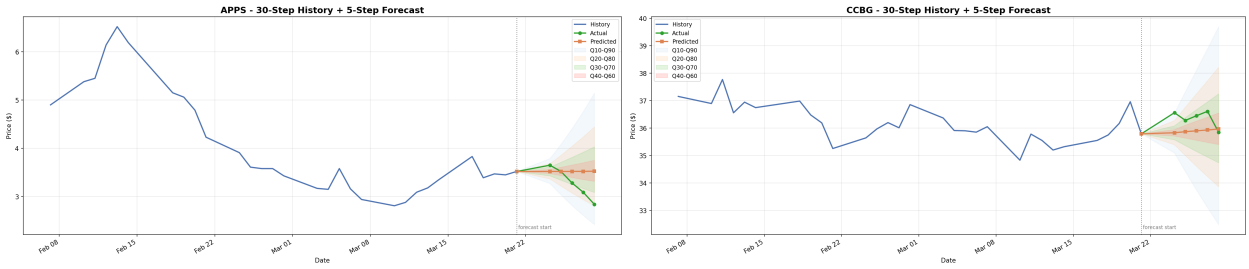

Figure 7: Chronos 2 forecast examples for APPS and CCBG.

Chronos T5-Large Chronos T5-Large [5, 7] was evaluated for this task. It achieved the highest reported directional accuracy (62.9%) among all evaluated models. Table 6 summarizes the results.

Metric	Std. Scale
MAE	1.2130
RMSE	1.3669
MAPE	2.63%
sMAPE	2.61%

Table 6: Chronos T5-Large Metrics.

4.6 Graph Models

Equities do not evolve independently — firms in the same sector co-move under shared macro shocks, and correlated stocks transmit information through supply chains and factor exposures. To represent this, the stock universe was encoded as a daily graph: each node is a stock; static sector edges fully connect same-sector firms; dynamic correlation edges are rebuilt from rolling 20-day Pearson correlations of returns (top-10 neighbours per stock, symmetrized). Node features combine seven scalars (log return, HL range, volume change, 5-day and 20-day momentum, news count, sentiment score) plus optional 768-dim FinBERT embeddings.

EvolveGCN EvolveGCN [8] evolves graph convolution parameters over time, allowing the message-passing operator to adapt to changing graph structure. This was well-suited to financial forecasting where inter-stock dependencies shift across regimes. It served as a principled dynamic-graph baseline for time-varying dependence patterns.

GATv2 The graph-attention architecture used GATv2 [10] in a late-fusion spatiotemporal design. A shared one-layer LSTM first encoded each stock's 20-day history of seven scalar features into a 128-dimensional temporal state. FinBERT embeddings (768-dim), when enabled, were concatenated after this encoding step, producing a late-fusion node representation projected to 256 dimensions.

Formally, for stock i on day t:

$$\begin{aligned} h_{i,\text{temp}}^{(t)} &= \text{LSTM}\Big(X_i^{(t)}\Big) \in \mathbb{R}^{128}, \\ z_i^{(t)} &= \Big[h_{i,\text{temp}}^{(t)} \parallel b_i^{(t)}\Big], \\ h_i^{(0)} &= \text{Dropout}\Big(\text{GELU}\Big(W_z\,\text{LN}(z_i^{(t)}) + c_z\Big)\Big). \end{aligned}$$

Two GATv2Conv layers followed: the first used four heads with concatenation (256-dim); the second averaged four heads to a 64-dim bottleneck, then projected back to 256 with a residual connection. Edge weights from sector and correlation graphs were summed and passed as attention priors.

The MLP head produced one scalar prediction per stock, targeting cumulative H-day log return. The training objective combined Huber loss (magnitude stability) with a cross-sectional Pearson-correlation term (ranking alignment):

$$\begin{aligned} \mathcal{L} &= \mathcal{L}_{\text{Huber}} + \mathcal{L}_{\text{Pearson}}, \\ \mathcal{L}_{\text{Pearson}} &= 1 - \text{corr}\Big(\hat{\mathbf{y}}_{\text{valid}}^{(t)}, \mathbf{y}_{\text{valid}}^{(t)}\Big). \end{aligned}$$

Early stopping used validation Rank IC to keep model selection aligned with portfolio ranking rather than pointwise error.

HTGNN To model richer relational structure, a Heterogeneous Temporal Graph Neural Network (HTGNN) [11] was additionally considered. This architecture handles graphs with multiple node and edge types alongside explicit temporal evolution, making it a natural fit for multimodal equity forecasting where sector edges, correlation edges, and time-varying node features coexist.

5 Results and Findings

Across all evaluated settings, model performance depended strongly on both architectural choice and modality combination. Unimodal price-only models provided a useful reference but were consistently limited in capturing abrupt, sentiment-driven moves.

Adding the news modality produced the most reliable improvement. Bimodal fusion of time-series features with news-derived sentiment yielded better directional consistency and stronger cross-sectional ranking than purely temporal baselines, improving the model's ability to distinguish between stocks with stronger and weaker near-term return prospects. This gain was more consistent than those from adding SEC fundamentals alone, likely because news aligns naturally with the daily prediction horizon while accounting data updates at lower frequency.

The structured SEC modality provided useful complementary context. Rather than driving short-term predictions directly, it enriched the feature space with profitability, leverage, and liquidity measures that are difficult to infer from price action alone. This suggests accounting data is better suited for medium-horizon forecasting or risk conditioning than for immediate next-week return estimation.

Graph-based methods added a distinct type of value by capturing cross-sectional dependencies that sequence models cannot represent. Graph attention mechanisms propagated information between economically related firms, allowing models to respond to peer signals — particularly useful when the objective emphasizes relative ranking across the universe. However, added architectural complexity did not automatically improve performance; simpler fusion pipelines were often more robust than highly parameterized alternatives when joint data coverage constrained sample sizes.

The central empirical finding is that multimodal forecasting is beneficial when each modality is aligned to the prediction horizon and fused in a controlled way, with time-series plus news sentiment providing the clearest and most repeatable advantage.

6 Conclusion and Future Work

This report investigated multimodal financial time-series forecasting using the FinMultiTime dataset, comparing baseline, pretrained, and graph-based approaches for five-day log-return prediction. Combining heterogeneous inputs is a practical way to improve short-horizon forecasting beyond price history alone. Fusing daily market features with news sentiment produced the most consistent gains; graph-based architectures added value through inter-stock relational modelling.

A key methodological lesson is that successful multimodal forecasting depends as much on data construction and alignment as on model choice. Building clean daily samples required careful synchronization of time-series, news, and low-frequency SEC filings — making the dataset pipeline a substantial contribution in its own right.

Several directions remain open. First, the text modality could be improved by replacing aggregated sentiment scores with stronger contextual encoders or retrieval-based summarization. Second, SEC features could be extended with refined ratio engineering, filing surprise measures, or explicit restatement handling. Third, graph construction could learn time-varying relations from market behaviour rather than relying on fixed sector metadata. Finally, future evaluation should emphasize downstream portfolio metrics, transaction-cost-aware backtesting, and cross-regime robustness.

In summary, multimodal learning is a promising direction for financial forecasting when the design remains sensitive to the differing update frequencies, noise profiles, and economic meaning of each data source.

References

[1] W. Wenyan0110, "Multimodal Dataset (Image, Text, Table, Time Series) for Financial Time-Series Forecasting," Hugging Face dataset repository. [Online]. Available: dataset page. Accessed: Apr. 18, 2026.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *ICLR*, 2023.

[4] S.-A. Chen, C.-L. Li, N. Yoder, S. O. Arik, and T. Pfister, "TSMixer: An all-MLP architecture for time series forecasting," *TMLR*, 2023.

[5] A. F. Ansari et al., "Chronos: Learning the language of time series," *arXiv:2403.07815*, 2024.

[6] M. A. Alharthi et al., "Chronos-2: From univariate to universal forecasting," *arXiv:2503.12746*, 2025.

[7] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *JMLR*, vol. 21, no. 140, pp. 1–67, 2020.

[8] A. Pareja et al., "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *AAAI*, vol. 34, no. 04, pp. 5363–5370, 2020.

[9] P. Veličković et al., "Graph attention networks," in *ICLR*, 2018.

[10] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *ICLR*, 2022.

[11] Z. Fan et al., "Heterogeneous temporal graph neural network," in *KDD*, pp. 509–519, 2021.

[12] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," *arXiv:1908.10063*, 2019.